

The Nmap Playbook

A complete, detailed guide to network scanning and reconnaissance with Nmap — from host discovery to scripting and evasion

From a single ping sweep to full service enumeration, this playbook builds the command fluency, scan-type intuition, and NSE scripting reference that penetration testers and SOC analysts rely on for reconnaissance and defense.

Author: Swastik Sagar
Final-year Computer Science Undergraduate
SOC Analyst Intern

August 1, 2026

Preface

Nmap (Network Mapper) is the single most widely used tool for network discovery and security auditing, and for good reason: almost every reconnaissance workflow, vulnerability assessment, and network inventory task starts with an Nmap scan. It is free, open-source, cross-platform, and after more than two decades of development, remarkably complete — host discovery, port scanning, service and OS fingerprinting, and a full scripting engine are all built in.

This playbook walks through Nmap the way it is actually used in practice: starting with host discovery and basic port scans, then building up through service detection, OS fingerprinting, the Nmap Scripting Engine, timing and performance tuning, and firewall/IDS evasion, before covering output formats and a set of practical scanning recipes for real engagements. Every chapter pairs the concept with the exact command syntax needed to use it.

What this book covers

- Nmap fundamentals: syntax, target specification, and basic usage
- Host discovery techniques and ping sweep options
- Port scanning techniques: SYN, Connect, UDP, FIN/NULL/Xmas, and more
- Service and version detection
- OS fingerprinting
- The Nmap Scripting Engine (NSE): categories, usage, and key scripts
- Timing templates and performance tuning
- Firewall and IDS evasion techniques
- Output formats and integrating Nmap into larger workflows
- Practical scanning recipes for common SOC and pentest scenarios
- The wider ecosystem: Zenmap, Ncat, Ndiff, and alternatives like Masscan
- Legal and ethical considerations

Swastik Sagar

Contents

Preface	i
1 Introduction to Nmap	1
1.1 What Nmap is	1
1.2 Why it matters for security work	1
1.3 How a scan actually works, at a glance	1
2 Installation and Setup	2
2.1 Installing Nmap	2
2.2 Verifying the install	2
2.3 Privileges	2
3 Nmap Fundamentals	3
3.1 Basic syntax	3
3.2 Target specification	3
3.3 A first scan	3
3.4 Commonly used flags	4
4 Host Discovery	5
4.1 Why discover hosts first	5
4.2 Discovery techniques	5
5 Port Scanning Techniques	6
5.1 TCP Connect scan (-sT)	6
5.2 TCP SYN scan (-sS)	6
5.3 UDP scan (-sU)	6
5.4 Other TCP scan types	7
5.5 Port states Nmap reports	7
6 Service and Version Detection	8
6.1 The -sV flag	8
6.2 How it works	8

7	OS Detection	9
7.1	The -O flag	9
7.2	Combining detection flags	9
8	The Nmap Scripting Engine (NSE)	10
8.1	What NSE is	10
8.2	Script categories	10
8.3	Using NSE	10
8.4	Useful scripts to know	11
9	Timing and Performance	12
9.1	Timing templates	12
9.2	Fine-grained timing control	12
10	Firewall and IDS Evasion	13
10.1	Fragmentation	13
10.2	Decoys	13
10.3	Source spoofing and idle scanning	13
10.4	Other evasion options	14
11	Output Formats	15
11.1	Output flags	15
11.2	Working with XML output	15
12	Practical Scanning Recipes	16
12.1	Quick recon	16
12.2	Full TCP port enumeration	16
12.3	Standard service assessment	16
12.4	Vulnerability-focused scan	16
12.5	Web server enumeration	16
12.6	Stealthy scan of a monitored network	17
13	The Wider Nmap Ecosystem	18
13.1	Bundled tools	18
13.2	Related tools worth knowing	18
14	Legal and Ethical Considerations	19
14.1	Authorization first	19
14.2	Good practice for authorized engagements	19

Introduction to Nmap

1.1 What Nmap is

Nmap is an open-source tool for network discovery and security auditing. Given one or more targets, it determines which hosts are up, which ports are open on those hosts, what service and version is running behind each open port, and often what operating system the host is running — all without needing credentials or agents on the target.

1.2 Why it matters for security work

- **Asset discovery** — finding every live host on a network, including ones nobody remembered to document
- **Attack surface mapping** — enumerating exposed services before an attacker does
- **Vulnerability triage** — NSE scripts can flag outdated versions and known misconfigurations directly
- **Firewall/IDS validation** — scanning your own perimeter is one of the fastest ways to confirm rules actually do what you think
- **Incident response** — quickly fingerprinting a suspicious host's exposed services during an investigation

Legal reminder

Only scan networks and hosts you own or have explicit written authorization to test. Unauthorized scanning can be illegal in many jurisdictions even when no exploitation follows. Chapter 12 covers this in more detail.

1.3 How a scan actually works, at a glance

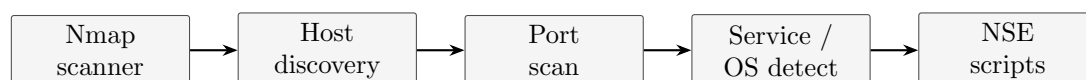


Figure 1.1. *The default Nmap pipeline: each stage narrows the target down and feeds the next.*

Any given scan can stop early (e.g. a bare host-discovery-only sweep) or chain all four stages together, which is what happens by default when you run Nmap against a target with no special flags beyond a port range.

Installation and Setup

2.1 Installing Nmap

Linux

```
sudo apt install nmap -- Debian/Ubuntu sudo dnf install nmap -- Fedora/RHEL sudo  
pacman -S nmap -- Arch
```

macOS

```
brew install nmap
```

Windows

-- Download the installer from nmap.org, which also installs -- Npcap (packet capture driver) and Zenmap (GUI front-end)

2.2 Verifying the install

Check version and basic sanity

```
nmap --version nmap -h | head -30
```

2.3 Privileges

Many of Nmap's most useful scan types (SYN scan, OS detection, most NSE scripts) require raw socket access and therefore root/Administrator privileges. Without elevated privileges, Nmap silently falls back to a full TCP Connect scan, which is slower and far noisier in logs.

Running with elevated privileges

```
sudo nmap -sS 192.168.1.1
```

Nmap Fundamentals

3.1 Basic syntax

General form

```
nmap [scan type(s)] [options] target specification
```

3.2 Target specification

Syntax	Meaning
192.168.1.1	A single IP address
192.168.1.0/24	CIDR notation, an entire subnet
192.168.1.1-50	An IP range (last octet 1 through 50)
192.168.1.1,5,10	A comma-separated list of hosts
scanme.nmap.org	A hostname, resolved via DNS
-iL targets.txt	Read targets from a file, one per line
-iR 100	Scan 100 random public hosts
--exclude 192.168.1.5	Exclude specific hosts from an otherwise broad scan

3.3 A first scan

Default scan: the 1000 most common TCP ports

```
nmap scanme.nmap.org
```

3.4 Commonly used flags

Flag	Purpose
-p	Specify ports, e.g. -p 80,443 or -p 1-65535 or -p- for all ports
-sV	Service/version detection
-O	OS detection
-A	Aggressive scan: OS detection, version detection, script scanning, and traceroute
-v / -vv	Increase verbosity
-oA	Output in all major formats at once
-T0--T5	Timing template, slowest to fastest

Host Discovery

4.1 Why discover hosts first

Scanning every port on every possible address in a large range wastes time. Host discovery (informally "ping scanning") quickly narrows a range down to hosts that actually respond, so later, more expensive scans only target live systems.

4.2 Discovery techniques

Flag	Technique
-sn	Ping scan only, no port scan (formerly -sP)
-PE	ICMP echo request
-PS	TCP SYN ping to specified ports (default 80)
-PA	TCP ACK ping
-PU	UDP ping
-PR	ARP ping, used automatically and reliably on a local Ethernet segment
-Pn	Skip host discovery entirely, treat all targets as online

Quick sweep of a subnet

```
nmap -sn 192.168.1.0/24
```

When to use -Pn

Many hosts and firewalls block ICMP, which would make them look offline to a default scan even though ports are actually reachable. -Pn forces Nmap to skip the discovery step and probe ports directly — essential when scanning behind a firewall that drops ping.

Port Scanning Techniques

5.1 TCP Connect scan (-sT)

Completes the full TCP three-way handshake using the OS's normal socket API. Reliable and doesn't require special privileges, but it's the noisiest option — every connection is logged like a real application connection because, from the target's perspective, it is one.

5.2 TCP SYN scan (-sS)

The default and most popular scan type when running as root. Sends a SYN packet and reads the response without completing the handshake:

Response	Nmap reports
SYN/ACK	Port is open (RST sent back to abandon the handshake)
RST	Port is closed
No response (after retries)	Port is filtered
ICMP unreachable	Port is filtered

Because the handshake is never completed, this scan is often called a **half-open scan** — faster and quieter than a full connect scan, since many applications never see the connection at all.

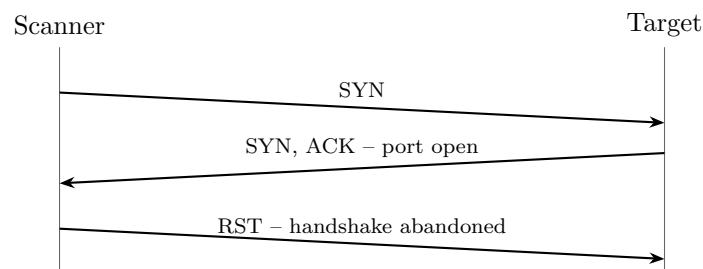


Figure 5.1. *The SYN scan never completes the handshake, which is what makes it faster and stealthier than a connect scan.*

5.3 UDP scan (-sU)

Sends UDP packets to each target port. Because UDP has no handshake, results are inherently ambiguous: an open port often produces no reply at all, so Nmap infers **open|filtered** unless a service replies or an ICMP port-unreachable error explicitly marks the port **closed**. UDP scans are much slower than TCP scans for this reason.

5.4 Other TCP scan types

Flag	Technique
-sF	FIN scan — sends a lone FIN flag; closed ports reply RST, open ports typically stay silent
-sN	NULL scan — sends a packet with no flags set at all
-sX	Xmas scan — sends FIN, PSH, and URG flags together (“lit up like a Christmas tree”)
-sA	ACK scan — doesn’t determine open/closed, only used to map firewall rule sets (stateful vs. stateless)
-sW	Window scan — variant of ACK scan that examines the TCP window size in the RST response
-sM	Maimon scan — FIN/ACK combination, exploits a quirk in some older BSD-derived stacks

Why FIN/NULL/Xmas exist

These scans exploit RFC 793 behavior: a closed port must respond with a RST, while an open port is supposed to silently drop packets that don’t have a SYN flag. Because they never send a SYN, some stateless firewalls and packet filters let them straight through, making them useful for evading simple filtering rules. Modern stateful firewalls generally aren’t fooled.

5.5 Port states Nmap reports

State	Meaning
open	An application is actively accepting connections on this port
closed	Port is reachable but no application is listening
filtered	A firewall or filter is blocking Nmap’s probes, so state can’t be determined
unfiltered	Port is reachable but Nmap can’t tell if it’s open or closed (seen with ACK scan)
open filtered	Nmap can’t tell if the port is open or filtered (common with UDP)
closed filtered	Nmap can’t tell if the port is closed or filtered (rare, seen with IP ID idle scans)

Service and Version Detection

6.1 The -sV flag

Once a port is known to be open, Nmap can probe it further to identify the actual application and version running there, rather than assuming based on the port number alone (which is easily misleading — any service can be bound to any port).

Version detection

```
nmap -sV 192.168.1.10 nmap -sV --version-intensity 9 192.168.1.10 -- try harder  
(0-9, default 7) nmap -sV --version-light 192.168.1.10 -- faster, fewer probes
```

6.2 How it works

Nmap sends a series of probes from its `nmap-service-probes` database and compares the responses against known signatures for hundreds of services, extracting product name, version number, and sometimes extra info like hostname or running modules (e.g. an Apache install's loaded PHP version).

Why this matters for vulnerability triage

Version strings are often the fastest way to spot an outdated, vulnerable service during an assessment — pairing `-sV` output with a CVE lookup is one of the most common first steps in a pentest or vulnerability scan.

OS Detection

7.1 The -O flag

Nmap can often fingerprint a target's operating system by analyzing subtle differences in how different TCP/IP stacks respond to a series of crafted probes — initial TTL values, TCP window sizes, options ordering, and how a host responds to malformed packets all vary by OS and even by version.

OS detection

```
sudo nmap -O 192.168.1.10 sudo nmap -O --osscan-guess 192.168.1.10 -- more  
aggressive guessing
```

Accuracy caveats

OS detection needs at least one open and one closed port to work reliably, and results are only ever a best-guess match against Nmap's fingerprint database — heavily firewalled hosts or unusual network stacks can produce no match or a wrong one.

7.2 Combining detection flags

The aggressive scan

```
sudo nmap -A 192.168.1.10 -- Equivalent to: -O -sV -sC --traceroute
```

The Nmap Scripting Engine (NSE)

8.1 What NSE is

The Nmap Scripting Engine lets Nmap run Lua scripts against scan targets, extending it far past pure port scanning into vulnerability detection, deeper enumeration, and even limited exploitation. Hundreds of scripts ship with Nmap by default, organized into categories.

8.2 Script categories

Category	Purpose
auth	Attempts to bypass or test authentication
broadcast	Discovers hosts via broadcast/multicast queries
brute	Brute-force credential guessing against services
default	Safe, fast scripts run automatically with <code>-sC</code> or <code>-A</code>
discovery	Deeper enumeration of hosts, services, and shared resources
dos	Tests for denial-of-service conditions (use with caution)
exploit	Attempts to exploit a known vulnerability
intrusive	May crash services or use significant resources; not run by default
malware	Checks whether a target is infected with common malware/backdoors
safe	Won't crash services or use excessive resources
vuln	Checks for specific known vulnerabilities

8.3 Using NSE

Common NSE usage

```
nmap -sC 192.168.1.10 -- run default scripts
nmap --script vuln 192.168.1.10 -- run all vuln-category scripts
nmap --script http-title 192.168.1.10 -- run one named script
nmap --script "http-*" 192.168.1.10 -- run all scripts matching a pattern
nmap --script-help http-title -- read a script's documentation
```

8.4 Useful scripts to know

Script	What it does
http-title	Grabs the title of a web page on an HTTP service
http-enum	Enumerates common directories/files on a web server
smb-os-discovery	Identifies OS/domain info via SMB
ssl-cert	Extracts and displays SSL/TLS certificate details
dns-brute	Brute-forces subdomains via DNS
ftp-anon	Checks whether anonymous FTP login is allowed
vulners	Cross-references detected versions against known CVEs

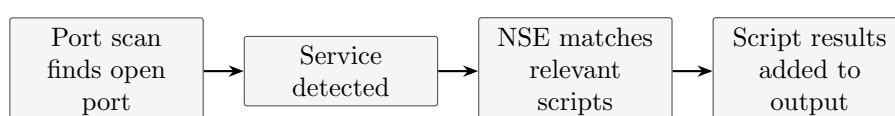


Figure 8.1. *NSE scripts are matched to a port automatically based on the service Nmap detects there.*

Timing and Performance

9.1 Timing templates

Template	Behavior
-T0	Paranoid — extremely slow, one probe at a time, for maximum IDS evasion
-T1	Sneaky — slow, intended to avoid IDS detection
-T2	Polite — slows down to use less bandwidth and target resources
-T3	Normal — the default, no special adjustment
-T4	Aggressive — faster scan, assumes a reasonably fast and reliable network
-T5	Insane — fastest, sacrifices accuracy for speed, easy to detect

9.2 Fine-grained timing control

Manual timing options

```
nmap --min-rate 1000 target -- send at least 1000 packets/sec
nmap --max-retries 2 target -- limit retransmissions
nmap --host-timeout 30m target -- give up on a slow host after 30 minutes
nmap -T4 --min-parallelism 100 target
```

Choosing a template

-T4 is the most common choice for scans on networks you control — fast without being reckless. -T0/-T1 matter almost exclusively for deliberate evasion during authorized red-team engagements, where speed is traded for staying under an IDS's detection threshold.

Firewall and IDS Evasion

10.1 Fragmentation

Packet fragmentation

```
nmap -f target -- fragment packets into 8-byte chunks nmap -f -f target --  
fragment into 16-byte chunks nmap --mtu 24 target -- set a custom fragment size  
(multiple of 8)
```

Splitting probes into fragments can slip past simple packet filters that don't reassemble fragments before inspecting them, though most modern firewalls handle reassembly correctly.

10.2 Decoys

Decoy scanning

```
nmap -D RND:10 target -- generate 10 random decoy source IPs nmap -D  
decoy1,decoy2,ME target -- mix specific decoys with the real scan
```

The target sees probes from many source addresses at once, making it harder for a defender to know which one is the real scanner.

10.3 Source spoofing and idle scanning

Source manipulation

```
nmap -S spoofed_ip target -- spoof the source address nmap -e eth0 target --  
specify the outgoing interface nmap -g 53 target -- use a fixed source port (e.g.  
53, often trusted) nmap -sI zombie_host target -- idle/zombie scan, fully hides  
the scanner's IP
```

Idle scanning in one sentence

An idle scan uses a third-party "zombie" host's predictable IP ID sequence to infer port states on the real target without ever sending it a single packet from the scanner's own address — clever, but slow and dependent on finding a suitable zombie host.

10.4 Other evasion options

Flag	Purpose
<code>--data-length</code>	Append random data to make packets look less like a scan
<code>--randomize-hosts</code>	Scan targets in random order instead of sequentially
<code>--spoof-mac</code>	Spoof the source MAC address on a local segment
<code>--badsum</code>	Send packets with an invalid checksum, to see what slips through

Blue-team note

Every technique in this chapter has a matching detection opportunity: fragment reassembly logging, IP ID anomaly detection, and NetFlow analysis for decoy/many-source patterns are all standard SOC controls that specifically target these evasion methods.

Output Formats

11.1 Output flags

Flag	Format
-oN file	Normal output, human-readable, matches terminal output
-oX file	XML output, ideal for parsing programmatically
-oG file	Grepable output (legacy, but still handy for quick <code>grep/awk</code> pipelines)
-oA basename	Writes all three formats at once using one base filename
-oS file	"Script kiddie" output, a novelty format

Typical usage

```
nmap -sV -oA scan_results 192.168.1.0/24 -- produces scan_results.nmap,  
scan_results.xml, scan_results.gnmap
```

11.2 Working with XML output

XML output is the preferred format for feeding Nmap results into other tooling — SIEM ingestion, custom dashboards, or converting to HTML reports.

Convert XML to an HTML report

```
xsltproc scan_results.xml -o scan_results.html
```

Practical Scanning Recipes

12.1 Quick recon

Fast initial sweep of a subnet

```
nmap -sn 192.168.1.0/24
```

12.2 Full TCP port enumeration

All 65535 TCP ports, then follow up on the open ones

```
nmap -p- -T4 -oN allports.txt 192.168.1.10 nmap -sV -sC -p <open_ports_here> 192.168.1.10
```

12.3 Standard service assessment

Version + default scripts + OS, common assessment baseline

```
sudo nmap -sV -sC -O -oA assessment 192.168.1.10
```

12.4 Vulnerability-focused scan

Run the vuln script category against known open ports

```
nmap --script vuln -p 21,22,80,443,445 192.168.1.10
```

12.5 Web server enumeration

Common web-focused workflow

```
nmap -p 80,443 --script http-enum,http-title,ssl-cert 192.168.1.10
```

12.6 Stealthy scan of a monitored network

Slower, quieter, still thorough

```
sudo nmap -sS -T2 -f --data-length 24 -p- 192.168.1.10
```

The Wider Nmap Ecosystem

13.1 Bundled tools

Tool	Purpose
Zenmap	Official GUI front-end for Nmap; good for visualizing topology and saving scan profiles
Ncat	Modern, feature-rich replacement for netcat; supports SSL, proxying, and connection brokering
Ndiff	Compares two Nmap XML scans and reports what changed between them
Nping	Custom packet generation and response analysis, useful for testing and troubleshooting

13.2 Related tools worth knowing

Tool	Notes
Masscan	Extremely fast asynchronous port scanner, built for internet-scale scans; less detailed than Nmap per host
RustScan	Fast initial port discovery, commonly piped into Nmap for detailed follow-up scanning
Shodan	Internet-wide scan search engine; useful for passive recon without touching the target directly

A common fast-then-thorough workflow

```
masscan -p1-65535 192.168.1.10 --rate 1000 -oL masscan.out nmap -sV -sC -p
<ports_from_masscan> 192.168.1.10
```

Legal and Ethical Considerations

14.1 Authorization first

Scanning a network you don't own or don't have explicit written permission to test can violate computer misuse laws in many countries, regardless of intent — this applies even to seemingly harmless discovery scans. Always work from a signed scope of work or rules of engagement document for any assessment.

14.2 Good practice for authorized engagements

- Confirm scope in writing: which hosts, which ports, which time windows
- Keep a timestamped log of every scan run, for later correlation with any alerts raised
- Notify relevant stakeholders (network/security teams) before intrusive or **dos**-category scripts
- Avoid **intrusive** and **dos** NSE categories on production systems unless specifically scoped
- Store and share scan output securely — it's a map of the target's attack surface

Scanning your own home lab

None of this applies to systems you personally own and control end-to-end (a home lab, your own VM range) — that's exactly the environment to practice every technique in this book safely.